머신러닝 핵심 정리 핸드북

Iris 데이터셋 기반 탐색적 데이터 분석, 알고리즘 구현 및 성능 최적화

1. 머신러닝의 시작 & Iris 데이터 탐색 (EDA)

> **개념 요약:** Iris(붓꽃) 데이터셋은 세 가지 품종(setosa, versicolor, virginica)을 4개의 특성(꽃받침/꽃잎의 길이와 너비)으로 분류하는 머신러닝의 대표적인 표준 데이터셋입니다.

1.1 Iris 데이터 로드 및 데이터프레임 구성

```
from sklearn.datasets import load_iris
import pandas as pd

# Iris 데이터셋 로드
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target
df['species'] = df['target'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})

# 데이터 상위 5개 및 정보 확인
print(df.head())
print(df.info())
```

1.2 탐색적 데이터 분석 (EDA) 및 시각화

```
import matplotlib.pyplot as plt
import seaborn as sns

# 페어플롯: 특성 간의 관계 및 품종별 분포 확인
sns.set_theme(style='whitegrid')
sns.pairplot(df, hue='species', palette='husl')
plt.show()

# 상관관계 히트맵
plt.figure(figsize=(6, 4))
numeric_df = df.drop(columns=['target', 'species'])
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()
```

1.3 베이스라인 모델 (Zero-R / Feature가 없을 때)

특성 없이 최빈값(Most Frequent) 분포만으로 예측할 경우, 균등하게 50개씩 분배된 Iris 데이터의 기본 정확도는 **33.33% (1/3)**가 됩니다.

```
from sklearn.dummy import DummyClassifier

dummy_clf = DummyClassifier(strategy='most_frequent')
dummy_clf.fit(df[iris.feature_names], df['target'])
print(f"베이스라인 정확도: {dummy_clf.score(df[iris.feature_names], df['target']) * 100:.2f}%")
```

2. 결정 트리 (Decision Tree) & 과적합 (Overfitting)

> **과적합(Overfitting)이란?** 모델이 훈련 데이터의 노이즈까지 과하게 암기하여, 새로운 테스트 데이터에 대해 성능이 떨어지는 현상입니다. 트리가 깊어질수록 과적합 위험이 증가합니다.

2.1 Train / Test 분할 및 결정 트리 학습

훈련 데이터로만 성능을 검증하는 것은 "시험 문제를 외우고 응시하는 것"과 같으므로 반드시 검증용 테스트 세트를 분리해야 합니다.

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# 데이터 분할 (8:2 비율, 계층적 샘플링)
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.target
)

# 결정 트리 생성 및 학습 (max_depth 제어로 과적합 방지)
dt_clf = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_clf.fit(X_train, y_train)

# 정확도 평가
y_pred = dt_clf.predict(X_test)
print(f"결정 트리 정확도: {accuracy_score(y_test, y_pred) * 100:.2f}%")
```

2.2 트리 구조 시각화 및 교차 검증 (Cross-Validation)

```
# 트리 시각화
plt.figure(figsize=(10, 6))
plot_tree(dt_clf, feature_names=iris.feature_names, class_names=iris.target_names,
filled=True)
plt.show()

# 5-Fold 교차 검증 (데이터 편향 방지)
from sklearn.model_selection import cross_val_score
scores = cross_val_score(dt_clf, iris.data, iris.target, cv=5)
print(f"교차 검증 평균 정확도: {scores.mean() * 100:.2f}%")
```

2.3 GridSearchCV를 활용한 하이퍼파라미터 튜닝

```
from sklearn.model_selection import GridSearchCV

param_grid = {'max_depth': [1, 2, 3, 4, 5], 'min_samples_split': [2, 3, 4]}
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=5)
grid_search.fit(X_train, y_train)
print(f"최적 파라미터: {grid_search.best_params_}")
```

3. 오탐지 구조 & 성능 평가 지표 (Confusion Matrix)

> **평가 지표의 핵심 개념:**
> • **Precision (정밀도):** 모델이 '참'이라고 예측한 것 중 진짜 '참'인 비율 (1종 오류 억제)
> • **Recall (재현율):** 실제 '참'인 데이터 중 모델이 '참'으로 맞춘 비율 (2종 오류 억제)
> • **F1-Score:** 정밀도와 재현율의 조화 평균 지표

3.1 혼동 행렬 및 상세 분류 보고서 측정

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# 혼동 행렬 생성
conf_matrix = confusion_matrix(y_test, y_pred)

# 시각화
plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

# 상세 분류 보고서 출력
print("=== Classification Report ===")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

4. 회귀(Regression) 모델에서의 규제 (L1 Lasso / L2 Ridge)

다항식 회귀에서 차수가 너무 높아지면 뱀처럼 꺾이는 과적합 현상이 발생합니다. L1, L2 규제는 계수 크기를 제한하여 곡선을 부드럽게 만들어 줍니다.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.pipeline import make_pipeline

# 8차 다항 회귀 + 규제 모델 비교
degree = 8
linear_model = make_pipeline(PolynomialFeatures(degree), LinearRegression()) # 과적합 발생
ridge_model = make_pipeline(PolynomialFeatures(degree), Ridge(alpha=1.0))     # L2 규제
lasso_model = make_pipeline(PolynomialFeatures(degree), Lasso(alpha=0.1))     # L1 규제
```

5. 앙상블 기법 (Ensemble): 배깅 vs 부스팅

구분	배깅 (Bagging, 예: Random Forest)	부스팅 (Boosting, 예: XGBoost, LightGBM)
학습 방식	병렬 독립 학습 (Parallel)	순차적 보완 학습 (Sequential)
핵심 원리	무작위 추출 트리들의 다수결 투표	이전 트리의 오답(잔차)에 가중치 부여
주요 목적	분산 감소 (과적합 방지 및 안정화)	편향 감소 (예측 정확도 극대화)

5.1 랜덤 포레스트 (Random Forest) 구현

```
from sklearn.ensemble import RandomForestClassifier

rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train, y_train)
print(f"Random Forest 정확도: {rf_clf.score(X_test, y_test) * 100:.2f}%")
```

5.2 XGBoost & LightGBM 구현

```
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

# XGBoost
xgb_clf = XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42,
eval_metric='mlogloss')
xgb_clf.fit(X_train, y_train)

# LightGBM
lgb_clf = LGBMClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
lgb_clf.fit(X_train, y_train)

print(f"XGBoost 정확도: {xgb_clf.score(X_test, y_test) * 100:.2f}%")
print(f"LightGBM 정확도: {lgb_clf.score(X_test, y_test) * 100:.2f}%")
```

6. 모델 성능 통합 비교 & Optuna 자동 튜닝

6.1 모델별 성능 통합 비교 파이프라인

```
import time
import pandas as pd

models = {
    "Decision Tree": dt_clf,
    "Random Forest": rf_clf,
    "XGBoost": xgb_clf,
    "LightGBM": lgb_clf
}

results = []
for name, model in models.items():
    start = time.time()
    model.fit(X_train, y_train)
    t = time.time() - start
    acc = accuracy_score(y_test, model.predict(X_test))
    results.append({"Model": name, "Accuracy (%)": round(acc * 100, 2), "Time (s)": round(t,
4)})

print(pd.DataFrame(results))
```

6.2 Optuna를 활용한 베이지안 하이퍼파라미터 최적화

> **Optuna의 장점:** TPE(베이지안 최적화) 기반 알고리즘을 사용하여 기존 GridSearch 대비 훨씬 빠르게 고성능 하이퍼파라미터 조합을 탐색합니다.

```
import optuna

def objective(trial):
    # 하이퍼파라미터 탐색 범위 설정
    n_estimators = trial.suggest_int('n_estimators', 10, 200)
    max_depth = trial.suggest_int('max_depth', 2, 20)
    min_samples_split = trial.suggest_int('min_samples_split', 2, 10)

    model = RandomForestClassifier(
        n_estimators=n_estimators, max_depth=max_depth,
        min_samples_split=min_samples_split, random_state=42
    )
    model.fit(X_train, y_train)
    return accuracy_score(y_test, model.predict(X_test))

# Optuna 최적화 실행 (50회 탐색)
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)

print(f"최적 하이퍼파라미터: {study.best_params}")
print(f"최고 정확도: {study.best_value * 100:.2f}%")
```